Informe técnico · TPI Food Store · Segunda entrega (Unidades 4 a 8)

Bases de Datos II · Grupo 11 · Ciro Cattáneo · Ignacio Ceballos · Santiago Copia · Franco Gagliardi · Emmanuel Miranda · Ramiro Quiroga

Repositorio: `github.com/nachoceba/TP_Concurrencia_IA_Grupo11`. **Motores:** PostgreSQL 16+ (en los equipos del grupo, 17.10 y 18.6), MongoDB 8.0 y SQLite 3.37+. **Base del proyecto:** `parcial_BD/01_schema.sql` → `04_objects.sql` → `02_data_demo.sql` (demo) o `03_data_masivo.sql` (200.000 pedidos).

Regla de trabajo común a todas las unidades (`protocolo_seguridad.md`): nada se ejecuta sobre la base real. Se trabaja sobre un clon (`createdb -T`), se prueba primero dentro de `BEGIN … ROLLBACK` y se toma un respaldo antes de cada cambio estructural.

1. Capa de datos del proyecto

Motor	Qué guarda	Quién manda
PostgreSQL	`categoria`, `producto`, `usuario`, `pedido` y `detalle_pedido`. Las reglas viven en CHECK, FK y triggers de subtotal y total, en `sp_crear_pedido` con `FOR UPDATE`, en las vistas y la MV, y en los roles	**Fuente de verdad** de precio, stock, usuarios y ventas
MongoDB (U7)	Colección `pedido`: un documento por pedido, con las líneas y una copia mínima del cliente embebidas, y un validador R1/R2	Modelo de **lectura** derivado de PostgreSQL. Si hay diferencia, gana PostgreSQL
SQLite (U8)	Tablet del food truck: `producto_cache` (copia), y `pedido_borrador`, `item_borrador` y `cola_sync` (propuestas)	**El móvil propone, el servidor dispone**: la venta la decide PostgreSQL

2. Qué se implementó, cómo se probó y qué resultó

Unidad 4 · Normalización avanzada y redundancia — `Unidad 4/TP 6/`

- **Implementado.** Esquema heredado `control_lote_almacen`, llevado a FNBC (`tp_fnbc_control_lote.sql`). Las DF reales son {Lote, Depósito} → Responsable y Responsable → Depósito; las claves candidatas, {L, D} y {L, R}. Como Responsable → Depósito tiene un determinante que no es superclave, se descompone en `responsable_deposito` (PK: responsable) y `control_lote` (PK: lote + responsable), con la vista de compatibilidad `v_control_lote_almacen`. Para el panel se aplicó desnormalización controlada: la tabla resumen `resumen_venta_categoria_dia`, mantenida por el trigger `trg_resumen_venta_categoria_dia` (`tp_desnormalizacion_top_categorias.sql`). En `Unidad 4/TP 5/` está la vista materializada `mv_top_productos_categoria`, con índice único. La **migración de datos** está especificada en Kiro (`Unidad 4/TP 6/migracion_fnbc/.kiro/specs/migracion-fnbc-control-lote/`, 5 requisitos EARS) y tiene dos scripts transaccionales: la ida (`01_migracion_fnbc.sql`) y el **plan de reversión** (`02_reversion_fnbc.sql`).
- **Cómo se probó.** `prueba_migracion.sh` corre seis escenarios sobre bases descartables:
  - La ida valida la DF R → D **sobre los datos reales** antes de tocar nada, migra con `INSERT … SELECT` y verifica con `EXCEPT` en los dos sentidos. Cualquier diferencia deshace la transacción entera.
  - La tabla heredada no se borra: queda como `_legado` para poder volver.
  - La vuelta sincroniza el legado con lo cargado después de migrar. La vista reconstruye exactamente las 3 filas originales (501/30/801, 502/30/801, 503/31/802), lo que confirma la descomposición sin pérdida por el teorema de Heath. Se midió con `EXPLAIN (ANALYZE, BUFFERS)` sobre 200.000 pedidos, y una consulta de auditoría compara la tabla resumen contra el recálculo desde las tablas base.
- **Resultados de la migración** (`migracion_fnbc/evidencias/prueba_migracion.txt`):
  - **E1**, ida sin pérdida: 3 filas, 0 faltantes y 0 sobrantes.
  - **E2**, ya se puede dar de alta un responsable sin lote.

  - **E3**, la vuelta se **bloquea** si hay responsables sin lote, porque el esquema viejo no los puede guardar.
  - **E4**, la vuelta con lotes nuevos funciona sin pérdida (5 filas).
  - **E5**, si los datos violan la DF, la ida se rechaza y no cambia nada.
  - **E6**, la vuelta forzada se puede hacer, con advertencia.
  - Como costo conocido de la FNBC, la DF {L, D} → R no se preserva. Se aceptó y está documentado.
- **Resultados de la desnormalización.** La consulta del top 5 pasó de **1,26 ms y 1359 páginas a 0,099 ms y 6 páginas** (−92 %). La auditoría de sincronización devolvió 0 filas desincronizadas. La MV respondió en 0,087 ms, contra 242,7 ms de la consulta original. Las tres anomalías (inserción, borrado y actualización) desaparecen con el esquema descompuesto.

Unidad 5 · Administración y seguridad — `Unidad 5/TP6/`

- **Implementado.** `roles.sql`, con mínimo privilegio y spec Kiro en `docs/kiro_roles_spec/` (requisitos EARS por rol). Tiene 5 grupos NOLOGIN (`rol_app_lectura`, `rol_app_escritura`, `rol_soporte`, `rol_reportes`, `rol_auditor`) y 2 logins con límite de conexiones (`app_web` 20, `admin_datos` 5). La lectura se hace solo sobre vistas y la MV. La escritura, solo con `EXECUTE` de `sp_crear_pedido` (nunca `INSERT` directo). Soporte tiene privilegios por columna sobre `usuario`, sin `contrasena`. Se agregaron la vista `usuario_anon` (anonimización determinista), la auditoría del motor (`log_connections`, `log_disconnections`, `log_statement = 'mod'` y `log_line_prefix`) y `simulacro_incidente.sql`.
- **Cómo se probó.** Verificación en vivo el 15/10/2026 sobre la base efímera `tp6_u5_work`: consultas Q1 a Q5 a los catálogos (`pg_roles`, `has_*_privilege`), `auditoria_test.sql` con `ROLLBACK`, y el simulacro de acceso indebido (4 intentos fallidos, 1 acceso, lectura masiva como `rol_soporte` e intento de escalar a `admin_datos`). La IA reconstruyó el incidente “a ciegas” a partir del log anonimizado, y después se contrastó con lo que realmente pasó.
- **Resultados.** 7 roles creados, 12 grants verificados, `INSERT` directo denegado y escalamiento bloqueado (`permiso denegado para otorgar el rol "admin_datos"`; 0 filas nuevas en `pg_auth_members`). La verificación en vivo encontró 3 bugs en los scripts (una columna `fecha_alta` inexistente, una sintaxis inválida en el stub de autenticación y un `WHERE` sobre una columna sin privilegio), y los 3 se corrigieron. La reconstrucción de la IA acertó 3 afirmaciones, omitió 1 y tuvo **3 sin sustento** (una zona horaria mal convertida, un supuesto “ataque externo” desde una IP de documentación RFC 5737 y un `UPDATE` inventado). **Contención decidida por el equipo:** bloquear el origen en `pg_hba.conf`, verificar `pg_auth_members`, resetear las claves afectadas y volver a auditar los `GRANT`. **Auditoría activada en vivo** (`sql/simulacro_live.sh` y `docs/informe_auditoria_live.md`, 09/11/2026). Se aplicaron los parámetros con `ALTER SYSTEM` y el simulacro se reconstruyó desde el **log real del motor** (`docs/log_auditoria_live.txt`):
  - 4 `FATAL: password authentication failed` desde 127.0.0.1 y después un acceso con `scram-sha-256`.
  - Una lectura masiva de 6 usuarios, visible porque la ventana estaba en `'all'`.
  - `GRANT admin_datos` y `SET ROLE admin_datos` **denegados**, ahora con un login que no es superusuario.
  - 0 membresías nuevas.

  Contrastado con el log reconstruido de la Parte B, el texto real del error es otro en PostgreSQL 16, y cada intento es un PID distinto. Al activar la auditoría aparecieron dos **hallazgos**:
  - `log_statement` (`'mod'` o `'all'`) deja en el log las claves de `CREATE/ALTER ROLE … PASSWORD` en texto plano. **Decisión:** usar `\password`, o `SET log_statement = 'none'` en esa sesión.
  - `%u` registra el usuario de sesión y no el rol efectivo después de `SET ROLE`. **Propuesta:** pgaudit.

Unidad 6 · Mantenimiento — `Unidad 6/TP7/`

- **Implementado.** `SPEC_U6.md` con su `AGENTS.md`; `backup_foodstore.sh` (`pg_dump -Fc`, programado diario con cron, o con el Programador de tareas en Windows); `health_check.sql`; `maintenance.sql` (diagnóstico de tuplas muertas contra el umbral de autovacuum, y VACUUM/ANALYZE solo donde se justifica, nunca `VACUUM FULL`); `restore_test.sh` con `validacion_restore.sql`. Las contraseñas van en `pgpass`, nunca en el repo.
- **Cómo se probó.** Simulacro de desastre contra reloj el 28/10/2026: respaldo `foodstore_20261028_225715.dump`, restauración en `foodstore_restore_test` y validación de conteos, reglas, FK y triggers. Se probó en dos entornos (Linux con PostgreSQL 16 y Windows con PostgreSQL 18).
- **Resultados. RTO observado: 1,3 s** (`pg_restore` 0,7 s), contra 15 min previstos; con el seed masivo de 146 MB, 3,0 s; con `--jobs=4`, la mediana bajó de 3,3 a 2,7 s. **RPO:** 84 s de antigüedad del respaldo y pérdida efectiva 0; el diseño prevé 24 h. Conteos idénticos (6, 6, 17, 10 y 22), 0 filas inválidas y todas las FK

validadas. **Informe post-incidente** (`informe_u6.pdf`): el primer intento falló por un control que suponía un nombre de columna; se reemplazó por `pg_constraint.convalidated`. Además se corrigieron diferencias de Windows: la tabla `cliente`, que no existe en el esquema, la MV ausente y los `\r\n`. Mejoras: `restore_test.sh` semanal y archivado de WAL (PITR) para bajar el RPO.

Unidad 7 · NoSQL — `Unidad 7/TP 8/`

- **Implementado.** Colección `foodstore.pedido` con las líneas embebidas (producto y categoría) y una copia mínima del cliente, sin mail, celular ni contraseña. Importes en `Decimal128`. Validador `$jsonSchema` más `$expr`: R1 (subtotal = cantidad × precio) y R2 (total = suma de subtotales). Índice único en `id_pedido` y multikey en `lineas.producto.id_producto`. Los 7 pedidos y las 16 líneas se migraron con los datos reales del seed.
- **Cómo se probó.** Consultas a–d en `consultas.mongodb.js` y sus equivalentes en `consultas_postgresql.sql`. También se intentó insertar documentos que violan R1 y R2.
- **Resultados.** Los IDs coinciden en ambos motores: TERMINADO = {1, 2, 3, 4}; producto 11 = {1, 3, 4, 6}; total > 14.000 = {2, 3}. Los documentos inválidos se rechazaron enteros (*Document failed validation*), sin documentos parciales. La primera versión del modelo no coincidía con PostgreSQL y se corrigió (evidencia 04). **Decisión:** PostgreSQL sigue como sistema de registro, porque confirmar un pedido toca tres tablas y necesita FK y ACID. MongoDB queda solo como réplica de lectura si la carga de lectura lo justificara.

Unidad 8 · Móvil e integración — `Unidad 8/TP 9/`

- **Implementado.** Spec en formato Kiro (`.kiro/specs/food-store-feria/`, 6 requisitos EARS ligados a casos). Base SQLite de la tablet (`schema_sqlite.sql`): tablas STRICT, precios en centavos, CHECK y FK. Instantánea `v_snapshot_movil`, que exporta a CSV con las bajas lógicas como lápidas. `semaforo.sql` evalúa solo las solicitudes PENDIENTE. También se hizo el code review final de los scripts de la cursada.
- **Cómo se probó.** `pruebas_sqlite.sql` (COMMIT/ROLLBACK, C1 y C7) y `casos_2c.sh`, que prepara cada caso en la copia del servidor, exporta la instantánea, corre el semáforo y verifica PostgreSQL antes y después.
- **Resultados.**
  - C2: CONFLICTO, y verde después de aceptar el precio nuevo.
  - C3: RECHAZADO por stock.
  - C4: RECHAZADO por baja; la fila queda como lápida.
  - C5: SIMULADO_APROBADO.
  - C6: un UUID repetido da error de clave primaria y no se reevalúa.
  - **PostgreSQL sin cambios:** 7 pedidos, stock total 709 y la misma huella md5 antes y después.
  - **Code review:** H1, un bug real (`sp_crear_pedido` no podía vender las últimas unidades porque violaba `ck_producto_disponible_stock`), quedó corregido. H2 (sin idempotencia en el servidor) quedó aceptado como mejora. H3 (acumular stock en la tablet) se descartó con fundamento. H4 (un control de restore que nunca podía fallar) quedó corregido.

3. Pruebas: resumen

Tipo de prueba	Dónde	Resultado
Transacciones reversibles	Migración FNBC: ida y vuelta (U4), `auditoria_test.sql` (U5), pruebas de la tablet (U8)	6 escenarios de migración y reversión OK; un fallo deshace todo; `ROLLBACK` sin residuos
Copias de prueba	`createdb -T` en todas las unidades; `foodstore_restore_test`, `tp6_u5_work`, `food_store_tp8`, `food_store_review`	La base real nunca se tocó
Simulacro de acceso indebido	U5 · `simulacro_incidente.sql` + `simulacro_live.sh` (log real del motor)	4 FATAL + acceso + escalamiento denegado; reconstrucción desde el log; contención decidida por el equipo
Simulacro de desastre	U6 · `restore_test.sh`	**RTO 1,3 s** (masivo: 3,0 s); RPO previsto 24 h
Conflictos de sincronización	U8 · `casos_2c.sh`	C2 a C6 con el resultado esperado; PostgreSQL sin cambios

4. Decisiones de diseño

Decisión	Por qué	Salvaguarda
Descomponer `control_lote_almacen` en FNBC	La DF Responsable → Depósito causaba anomalías de inserción, borrado y actualización	Vista de compatibilidad, más migración en una transacción reversible
Conservar la tabla heredada como `_legado` y una vista con el nombre original	La vuelta no depende de un respaldo externo y la app no se rompe al migrar	La vuelta se bloquea si hay datos que el esquema viejo no puede representar
Tabla resumen con trigger para el top 5 del día (en vez de MV)	El panel necesita el dato en tiempo real, y una MV queda vieja hasta el `REFRESH`	Se puede reconstruir con un `SUM … GROUP BY`; hay una consulta de auditoría y se puede borrar sin perder datos
MV `mv_top_productos_categoria` para el reporte de gestión	Es un reporte analítico que tolera un día de atraso	Índice único para `REFRESH CONCURRENTLY` diario
Escribir solo con `sp_crear_pedido`	Una sola vía, que valida y bloquea stock con `FOR UPDATE`	`rol_app_escritura` no tiene `INSERT` directo
`log_statement = 'mod'` y no `'all'`	`'all'` genera demasiado volumen y expone datos personales en cada lectura	`'all'` solo durante la ventana de un incidente; claves solo con `\password`, porque el log las registraría
`pg_dump -Fc` diario más un restore de prueba	Un respaldo que no se restaura no está probado	Para RPO menor a 24 h hace falta WAL/PITR
MongoDB solo como modelo de lectura	El pedido toca tres tablas y necesita FK y ACID	Se regenera desde PostgreSQL
Tablet: copias con fecha, propuestas y semáforo	Sin señal no se puede decidir precio ni stock	`SIMULADO_APROBADO` no es una venta; UUID como clave de idempotencia

5. Uso de IA

Cada unidad tiene su bitácora o DUIA: `Unidad 4/TP 5/DUIA.md`, `Unidad 5/TP6/BITACORA.md`, `Unidad 7/TP 8/ia_consulta.md` y `Unidad 8/TP 9/docs/bitacora_ia.md`, con la DUIA. Las herramientas fueron OpenCode y Claude. En todas las unidades se mantuvo el mismo criterio: nada se aceptó sin ejecutarlo en una copia. Errores de la IA que el grupo detectó:

- `GRANT ALL` y una contraseña en texto plano en `roles.sql`.
- `RANDOM()` en la anonimización, que rompía la equivalencia.
- `log_statement = 'all'` permanente.
- `ANALYZE` antes del `REFRESH` de la MV.
- Un uso de `bc`, que Git Bash no tiene.
- Un modelo documental inicial con datos inventados.
- Tres afirmaciones sin sustento en la reconstrucción del incidente.